SMARTGALLERY — GUIA DE DESENVOLUPAMENT

Autor: Biel Matamala Pujol

Entorn: Python (Windows, VS Code)

Data: Octubre 2025

ÍNDEX

Introducció
Objectius del projecte
Requeriments tècnics
Estructura del projecte
Flux lògic del programa
Models d'IA i estratègia d'anàlisi
Llibreries i dependències
Configuració de l'entorn a VS Code
Depuració i proves
Possibles ampliacions
Notes de desenvolupament

1 Introducció

SmartGallery és una aplicació local escrita en Python que permet analitzar i organitzar grans conjunts de fotografies (RAW o JPG) mitjançant models d'intel·ligència artificial. L'objectiu principal és automatitzar el procés de classificació d'imatges dins de carpetes temàtiques (per exemple, "Persones", "Animals", "Paisatges", "Objectes", etc.).

El sistema utilitza un model d'IA preentrenat (com MobileNetV2 o EfficientNet) per reconèixer el contingut principal de cada imatge. Quan detecta un fitxer RAW, el converteix temporalment a JPG, el processa, i mou el fitxer RAW original a la carpeta corresponent segons la seva categoria detectada.

2 Objectius del projecte

Automatitzar la classificació d'imatges mitjançant IA.
Fer possible el treball 100% local, sense connexió a internet.
Acceptar fitxers RAW i JPG.
Crear una estructura d'organització clara dins de carpetes.
Ser ampliable: permetre substituir o entrenar models futurs.
Oferir una base per integrar més endavant una interfície visual (Streamlit).

3 Requeriments tècnics

Component	Descripció
Sistema operatiu	Windows 10 o superior
Entorn de desenvolupament	Visual Studio Code
Versió de Python	3.10+
Biblioteques principals	rawpy, imageio, tensorflow, os, shutil, streamlit
Entrades	Carpeta amb imatges RAW/JPG
Sortides	Carpeta amb subcarpetes per categoria d'imatge

4 Estructura del projecte

Estructura recomanada de carpetes i fitxers dins de VS Code:

```
smartgallery/
├── app.py
├── utils/
│   ├── raw_handler.py
│   ├── sorter.py
│   ├── logger.py
├── models/
│   └── classifier.py
├── data/
│   ├── input/
│   └── output/
└── docs/
    └── SmartGallery - Guia de Desenvolupament.docx
```

Descripció dels components:
app.py: punt d'entrada del programa.
utils/raw_handler.py: conversió RAW → JPG temporal.
utils/sorter.py: lògica de classificació i moviment de fitxers.
utils/logger.py: registre d'errors i operacions.
models/classifier.py: càrrega del model IA i inferència.
data/input: carpeta on es guarden les imatges originals.
data/output: carpeta de sortida amb les categories generades.

5 Flux lògic del programa

1. Llegeix la carpeta data/input/
2. Detecta el tipus de fitxer (RAW o JPG)
3. Converteix RAW → JPG temporal (si cal)
4. Analitza la imatge amb el model d'IA
5. Determina la categoria (persones, animals, etc.)
6. Mou el fitxer RAW original a la carpeta corresponent dins data/output/
7. Esborra el JPG temporal
8. Registra el procés al fitxer de log

6 Models d'IA i estratègia d'anàlisi

Models recomanats:

Model	Avantatges	Inconvenients
MobileNetV2	Lleuger i ràpid	Menys precís en escenes complexes
EfficientNetB0	Bon equilibri entre rendiment i precisió	Ligerament més lent
CLIP (OpenAI)	Pot analitzar conceptes generals	Més complex d'integrar

Estrategia inicial:
Començar amb MobileNetV2 ja preentrenat a ImageNet.
Ajustar categories personalitzades amb un mapeig senzill (p. ex., si detecta "dog", mou a /Animals/).

7 Llibreries i dependències

Llibreria	Ús	Comprovació
rawpy	Llegir fitxers RAW	import rawpy
imageio	Escriure JPG temporal	import imageio
tensorflow	Càrrega del model IA	import tensorflow
shutil	Moviment d'arxius	Inclosa a Python
os	Gestió de directoris	Inclosa a Python
streamlit	(Opcional) interfície visual	import streamlit

8 Configuració de l'entorn a VS Code

1. Crear entorn virtual:
   python -m venv venv
   venv\Scripts\activate

2. Instal·lar dependències:
   pip install rawpy imageio tensorflow streamlit

3. Configurar VS Code:
   Obrir carpeta del projecte.
   Seleccionar intèrpret .\venv\Scripts\python.exe.
   Instal·lar extensions: Python, Pylance, Code Runner.

9 Depuració i proves

Executar cada mòdul per separat (raw_handler.py, classifier.py).
Fer proves amb 3 o 4 imatges abans d'un conjunt gran.
Si hi ha errors, comprovar logs generats a logs.txt.
Fer servir print() o logging per validar el flux.

10 Possibles ampliacions

Afegir interfície visual amb Streamlit.
Guardar dades d'ús (categories més freqüents, temps de processament).

Permetre reentrenar el model amb imatges noves.
Integrar un sistema de busca intel·ligent d'imatges.
Connectar amb serveis com Google Photos o OneDrive.